

# Functional Verification of an AHB Protocol-Based Memory Using UVM

A  
CIAP  
for

Advanced Behavioral Design & Verification

*Submitted by*

DEVASHREE SURVE 124BTVL2010(40)



DEPARTMENT OF ELECTRONICS ( VLSI DESIGN & TECHNOLOGY)

SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE

(An Autonomous Institute Affiliated to University of Mumbai)

MUMBAI - 400 088, MAHARASHTRA (INDIA)

2024

# Functional Verification of an AHB Protocol-Based Memory Using UVM

## 1. Abstract

This project presents the verification of a simplified AHB-based memory module using the Universal Verification Methodology (UVM). The design under test (DUT) is a basic AHB slave memory with 16 locations of 8-bit data.

The verification environment includes standard UVM components such as sequence, driver, monitor, scoreboard, and coverage collector. Directed and random test scenarios were applied to validate read and write functionality.

The system achieved 100% functional coverage and 100% pass rate, indicating correct behavior for the tested scenarios.

However, the design uses a simplified AHB protocol (no pipelining, no response signals like HREADY/HRESP). Future improvements can include protocol completeness, constrained random enhancements, and corner-case stress testing.

## 2. Introduction

Advanced bus protocols such as Advanced High-performance Bus (AHB) are widely used in System-on-Chip (SoC) design. Verification of such protocols requires structured, scalable, and reusable methodologies to ensure functional correctness and reliability. In this project, a simplified AHB memory Design Under Test (DUT) is verified using a complete Universal Verification Methodology (UVM) testbench. The verification environment includes key components such as sequence generation, driving logic, monitoring, scoreboard-based checking, and functional coverage collection. Functional correctness is validated using a scoreboard, while coverage is measured using covergroups to ensure that all critical scenarios are exercised. The primary objective of this work is to validate read and write operations, ensure complete address space coverage, and detect mismatches automatically through self-checking mechanisms.

## 3. System Architecture

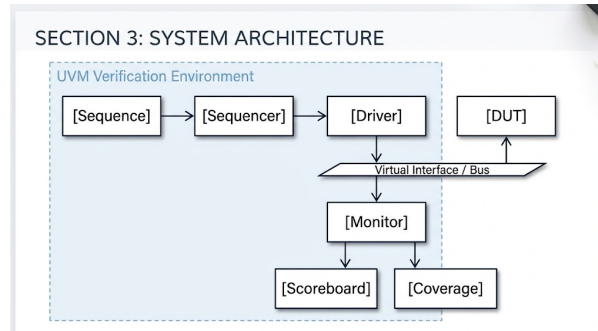


Figure 1: Block diagram of the standard UVM Verification Environment

## 4. Component-wise Explanation

### Transaction (ahb\_txn)

The transaction class represents a single AHB operation. It acts as the fundamental data packet that flows through the UVM verification components. It includes fields such as address (0–15), write data, read/write control signal, and read data from the DUT. This abstraction helps in separating stimulus generation from signal-level implementation.

### Sequence (ahb\_seq)

The sequence is responsible for generating stimulus for the DUT. It includes 16 write operations covering all memory addresses, followed by 16 read operations, and 20 random transactions. This ensures complete address coverage, validates both read and write functionality, and introduces basic stress testing through randomization.

### Driver (ahb\_driver)

The driver converts high-level transactions into pin-level signals for the DUT. It drives signals such as HADDR, HWDATA, HWRITE, HTRANS, and HSEL. However, the implementation is simplified and does not include handshake signals like HREADY or wait-state handling. Therefore, it performs functional-level driving rather than full protocol-compliant AHB transactions.

### Monitor (ahb\_monitor)

The monitor passively observes DUT signals and reconstructs transactions. It sends the reconstructed transactions through an analysis port to other components. For write operations, it directly captures the data, while for read operations, it waits one clock cycle to sample the HRDATA signal.

## **Scoreboard (ahb\_sb)**

The scoreboard maintains a reference memory model to verify correctness. For write transactions, it updates the model, and for read transactions, it compares the DUT output with expected values. The simulation results show PASS = 26 and FAIL = 0. However, the scoreboard only checks data correctness and does not verify protocol timing or handshake behavior.

## **Coverage (ahb\_cov)**

Functional coverage is implemented using covergroups. It tracks address coverage (0–15), read/write operations, and cross coverage between address and operation type. The achieved coverage is 100%. It is important to note that this is functional coverage only and does not include protocol or state-based coverage.

## **Environment (ahb\_env)**

The environment integrates all UVM components, including the driver, monitor, scoreboard, coverage collector, and sequencer. The driver is connected to the sequencer, while the monitor sends transactions to both the scoreboard and coverage components through analysis ports.

## **Test (ahb\_test)**

The test class initializes the environment and starts the sequence execution. It uses the UVM objection mechanism to control simulation flow, ensuring that the simulation runs until all transactions are completed.

## **DUT (ahb\_system\_dut)**

The Design Under Test is a simple memory module with 16 locations of 8-bit width. It performs write operations by storing data at a given address and read operations by returning stored data. The design is simplified and does not include advanced AHB features such as burst transfers, response signals, or pipelined operation.

## 5. Test Cases

The test scenarios are generated inside the sequence class (ahb\_seq). The verification methodology uses a combination of directed testing and random testing to ensure functional correctness and basic robustness of the design.

### Test Case 1: Sequential Write to All Addresses

#### Code Snippet:

```
for(int i=0;i<16;i++) begin
    tx = ahb_txn::type_id::create("wr");
    start_item(tx);
    tx.addr = i;
    tx.data = $urandom_range(1,255);
    tx.write = 1;
    finish_item(tx);
end
```

**Explanation:** This test performs write operations to all memory locations from address 0 to 15. Random data values are used to avoid constant-pattern bias and ensure realistic verification. This guarantees that every memory location is exercised at least once.

#### What it verifies:

- Memory write operation
- Address decoding correctness
- Data storage functionality

**Expected Result:** All addresses should correctly store the written data, and no mismatches should be reported by the scoreboard.

### Test Case 2: Sequential Read from All Addresses

#### Code Snippet:

```
for(int i=0;i<16;i++) begin
    tx = ahb_txn::type_id::create("rd");
    start_item(tx);
    tx.addr = i;
    tx.write = 0;
    finish_item(tx);
end
```

**Explanation:** This test reads all previously written memory locations. It verifies that the data stored during the write phase is retained and correctly returned during read operations.

**Scoreboard Logic:**

```
else if (mem_model[tx.addr] == tx.rdata)
    pass++;
```

**What it verifies:**

- Read functionality of DUT
- Data consistency
- Memory retention

**Expected Result:** The read data must match the stored data, and the PASS count in the scoreboard should increase.

**Test Case 3: Random Transactions****Code Snippet:**

```
repeat(20) begin
    tx = ahb_txn::type_id::create("rand");
    start_item(tx);
    assert(tx.randomize());
    finish_item(tx);
end
```

**Explanation:** This test introduces random read and write transactions. The address, data, and control signals are randomized to simulate unpredictable real-world access patterns.

**What it verifies:**

- Robustness of the DUT
- Mixed read/write operation handling
- Random access behavior

**Note:** Only 20 iterations are used, which provides limited stress. Industrial-level verification typically requires a much higher number of randomized transactions.

**Test Case 4: Mixed Read-After-Write Scenario**

**Description:** This scenario is implicitly tested through the sequence structure, where write operations are followed by read operations.

**Verification Flow:**

WRITE → STORE IN MODEL → READ → COMPARE

**Explanation:** This ensures that data written to memory is immediately available for read operations and verifies correct data propagation and timing behavior.

**What it verifies:**

- Read-after-write correctness
- Data propagation
- Timing consistency

## Summary of Test Cases

| Test ID | Test Case           | Purpose                                  |
|---------|---------------------|------------------------------------------|
| TC1     | Sequential Write    | Verify memory write across all addresses |
| TC2     | Sequential Read     | Verify correct data retrieval            |
| TC3     | Random Transactions | Test robustness and mixed operations     |
| TC4     | Read-After-Write    | Verify data consistency and timing       |

Table 1: Summary of Test Cases

## 6. Corner Cases

Corner cases represent extreme and boundary conditions that help identify hidden issues in a design. While the current verification setup covers some important scenarios, several critical cases remain untested due to the simplified nature of the DUT and testbench.

### Address Boundary Conditions

**Code Snippet:**

```
for (int i=0; i<16; i++)
```

**Explanation:** This test ensures that both the lowest address (0) and highest address (15) are accessed. It verifies correct memory indexing and ensures no overflow or boundary-related errors occur.

**Importance:**

- Detects address overflow issues
- Validates correct indexing

**Status:** Fully Covered

## Read After Write

### Monitor Logic:

```
if (vif.HWRITE)
    tx.data = vif.HWDATA;
else begin
    @(posedge vif.clk);
    tx.rdata = vif.HRDATA;
end
```

**Explanation:** This scenario ensures that data written to memory can be correctly read immediately afterward. It validates memory consistency and confirms that there is no data corruption.

**Status:** Covered

## Back-to-Back Transactions

### Driver Behavior:

```
@(posedge vif.clk);
vif.HADDR <= tx.addr;
...
@(posedge vif.clk);
vif.HSEL <= 0;
```

**Explanation:** Transactions are issued on every clock cycle without idle gaps. This tests the DUT's ability to handle continuous operations.

### Limitation:

- No pipelining
- No overlapping transactions

**Status:** Partially Covered

## Random Address Access

### Code Snippet:

```
assert(tx.randomize());
```

**Explanation:** Randomized transactions introduce variability in address and data patterns, helping uncover hidden bugs.

### Limitation:

- No constraints applied
- Limited randomness depth

**Status:** Weak Coverage



## Invalid Address Handling

### Code Definition:

```
rand bit [3:0] addr;
```

**Explanation:** The address is limited to 4 bits, restricting it to values between 0 and 15. Therefore, invalid addresses cannot be generated and tested.

**Status:** Not Covered

## Idle / No Transaction Condition

**Expected Scenario:** The DUT should remain idle when HSEL is low for an extended duration.

**Actual Behavior:** The driver continuously sends transactions without idle cycles.

### Impact:

- Idle behavior is not verified
- Power-saving or inactive states are untested

**Status:** Not Covered

## Protocol-Level Corner Cases

The following AHB protocol features are not implemented in the current design:

- Burst transfers
- HREADY wait states
- Error response (HRESP)
- Pipeline behavior

These limitations arise because the DUT implements a simplified version of the AHB protocol.

## Summary of Corner Cases

| Case | Description                              | Status            |
|------|------------------------------------------|-------------------|
| CC1  | Address Boundary (0–15)                  | Covered           |
| CC2  | Read After Write                         | Covered           |
| CC3  | Back-to-Back Transactions                | Partially Covered |
| CC4  | Random Access                            | Weak Coverage     |
| CC5  | Invalid Address Handling                 | Not Covered       |
| CC6  | Idle Condition                           | Not Covered       |
| CC7  | Protocol Features (Burst, HREADY, HRESP) | Not Covered       |

Table 2: Summary of Corner Cases

## 7. Simulation Results

```
UVM_INFO @ 0: reporter [RNTST] Running test ahb_test...
UVM_INFO /xcelium25.03/tools/methodology/UVM/CDNS-1.2/sv/src/base/uvm_object.svh(1271) @ 2035000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
UVM_INFO testbench.sv(187) @ 2035000: uvm_test_top.env.cov [COV] Coverage = 100.00 %
UVM_INFO testbench.sv(221) @ 2035000: uvm_test_top.env.sb [RESULT] PASS=26 FAIL=0
UVM_INFO /xcelium25.03/tools/methodology/UVM/CDNS-1.2/sv/src/base/uvm_report_server.svh(851) @ 2035000: reporter [UVM/REPORT/SERVER]
```

Figure 2: UVM simulation log showing successful execution of functional coverage

```
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    4
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[COV]      1
[RESULT]   1
[RNTST]    1
[TEST_DONE] 1

Simulation complete via $finish(1) at time 2035 NS + 59
/xcelium25.03/tools/methodology/UVM/CDNS-1.2/sv/src/base/uvm_root.svh:543    $finish;
```

Figure 3: The report summary indicates no warnings, errors, or fatal issues, confirming correct functional verification of the DUT.

The simulation was performed using the Cadence Xcelium tool on EDA Playground. The key results obtained from the simulation log are as follows:

- Coverage: 100%
- PASS: 26
- FAIL: 0
- Simulation Time: ~2035 ns

All generated transactions were executed successfully without any mismatches. The scoreboard reported zero failures, indicating that the DUT produced correct outputs for all test cases. Additionally, the functional coverage reached 100%, confirming that all defined coverage points were exercised during simulation.

However, it is important to note that the total number of transactions is relatively small (approximately 52), and no extensive stress testing was performed.

## 8. Waveform Analysis

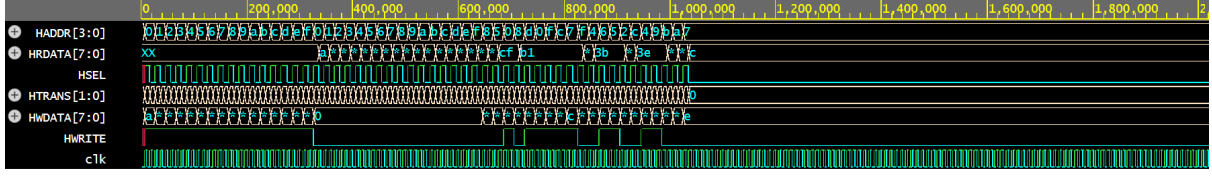


Figure 4: Simulation waveform illustrating AHB signal activity during write and read operations.

The waveform obtained from simulation illustrates the behavior of the AHB-based memory system during read and write operations. The key signals observed include HADDR, HWDATA, HRDATA, HWRITE, HSEL, HTRANS, and clk.

**Write Phase:** Initially, the address signal (HADDR[3:0]) increments sequentially from 0 to 15, indicating that all memory locations are accessed. During this phase, the write enable signal (HWRITE) is asserted high, and valid data is driven on HWDATA[7:0], which is stored in the memory.

**Read Phase:** After completing the write operations, the system transitions to the read phase where HWRITE is deasserted. The same addresses are accessed again, and the HRDATA[7:0] output matches the previously written values, confirming correct memory behavior.

**Initial Behavior:** At the start of the simulation, HRDATA shows undefined values (XX), which is expected due to uninitialized memory. Once write operations are performed, valid data appears in subsequent read cycles.

**Control Signals:** The HSEL signal remains active during valid transactions, indicating DUT selection. The HTRANS[1:0] signal stays at 2'b10, representing active transfers. The clock signal (clk) ensures proper synchronization of all operations.

**End of Simulation:** Towards the end, signal activity reduces and stabilizes, indicating completion of all transactions.

This confirms the functional correctness of the DUT for the applied test scenarios.

## 9. Observations

- Clean and modular UVM structure, easy to understand and extend
- Functional correctness verified with zero mismatches
- Achieved 100% functional coverage for defined scenarios
- Uses a simplified AHB protocol (not fully compliant)
- No assertion-based verification (SVA) implemented
- No negative or error-case testing performed
- Lacks timing and protocol-level validation
- Limited randomization, resulting in low stress testing

## 10. Future Scope

The current design can be further improved by adding protocol features such as `HREADY`, `HRESP`, and burst transfer support for better AHB compliance. Verification can be enhanced by incorporating SystemVerilog Assertions (SVA), constrained random testing, and increasing the number of test iterations. Additionally, extending the scoreboard for protocol checks, improving coverage with transition and sequence metrics, and including corner cases such as invalid addresses and reset conditions will make the verification more comprehensive.

## 11. Conclusion

This project demonstrates the verification of a simplified AHB-based memory system using UVM. The developed testbench successfully validates read and write operations, achieving 100% functional coverage with zero mismatches. The waveform results confirm correct data storage and retrieval.

However, the design is limited to functional-level verification and does not include full AHB protocol features such as handshake signals and burst transfers. Overall, the work provides a solid foundation for UVM-based verification and can be extended with advanced protocol support and verification techniques for improved completeness.